

Cerberus

A Two-Tier, Adaptive, Kernel-Level DDoS Mitigation System with
Flash-Crowd False-Positive Evaluation using eBPF/XDP

[Your Full Name]

[Your Student ID]

[Degree Programme — e.g., BSc (Hons) Computer Networks]

[University / Institution]

Supervisor: [Supervisor Name]

[Month, Year]

DRAFT — replace bracketed details, insert real citations, figures, and diagrams, and
remove the yellow author notes before submission.

Table of Contents

1. Chapter 1 — Introduction
2. Chapter 2 — Literature Review
3. Chapter 3 — Methodology
4. Chapter 4 — System Requirement Specification
5. Chapter 5 — Implementation
6. Chapter 6 — Testing and Evaluation
7. Chapter 7 — Concluding Remarks

(Regenerate a page-numbered table of contents in your word processor for the final version.)

Chapter 1 — Introduction

How to use this draft. Every paragraph is based on the system you actually built (Cerberus). Read each one, make sure you can explain it in your own words, and replace every **[INSERT REAL SOURCE / STAT]** marker with a genuine, verified reference before you submit. Do not submit any statistic or citation you have not personally checked.

1.1 Chapter Overview

This chapter introduces the research and explains why it was undertaken. It opens by describing what has been happening with distributed denial-of-service (DDoS) attacks and why, despite years of attention, they remain one of the most persistent threats to online services. From that broad picture the chapter narrows towards a specific and often overlooked weakness in the way current defences behave: their difficulty in telling a genuine attack apart from a sudden but entirely legitimate surge of real users, a situation usually described as a flash crowd. Having established the problem, the chapter presents a focused problem statement supported by recent evidence, separates the general implications of the problem from the specific shortcomings within the technical domain of this work, and closes that discussion by naming the research gap to be filled. It then sets out a single research question, the motivation that led to the study, the aim of the work, and a set of research-specific objectives. A rich picture of the proposed solution is given so that the reader can see how the parts of the system cooperate before any technical detail is introduced, and this is followed by the hardware and software resources used, a clear statement of what the project does and does not attempt, and a short summary linking the chapter to the rest of the dissertation.

1.2 Problem Background — what has actually happened

Almost every service that individuals and organisations depend on today is reachable over the Internet. Banking, government portals, online retail, streaming, healthcare

booking, and education all assume constant availability, and that assumption is exactly what makes availability such an attractive target. A distributed denial-of-service attack does not attempt to steal data or gain control of a machine; its single goal is to make a service unavailable to the people who legitimately want to use it. It achieves this by consuming some finite resource — network bandwidth, connection state, CPU cycles, or memory — faster than the victim can replenish it, until genuine requests can no longer be served.

What makes these attacks so difficult to counter is their distributed nature. Rather than originating from a single, easily blocked source, the traffic is spread across a large number of compromised machines, often ordinary home devices and, increasingly, poorly secured Internet-of-Things hardware that has been recruited into a botnet. Because the requests come from many places at once and each individual packet may be perfectly well-formed, the defender cannot simply look for malformed data or a single bad address. The challenge is not recognising an obviously broken packet but recognising an abnormal *pattern* assembled from otherwise ordinary traffic.

Over recent years the problem has grown rather than receded. Industry threat reports describe a steady rise in both the frequency of attacks and the peak volumes they reach, with record-setting volumetric floods now measured in terabits per second and short, high-intensity bursts becoming more common **[INSERT REAL STAT — e.g., the most recent Cloudflare DDoS Threat Report or NETSCOUT Threat Intelligence Report; quote the specific figure and its year]**. Two developments in particular have worsened the situation. The first is accessibility: attack tools and even rentable "booter" or "stresser" services have made launching a sizeable attack cheap and technically undemanding, lowering the barrier to entry dramatically. The second is architectural: many established defences operate at a distance from the machine being protected — in a dedicated scrubbing appliance, at the network edge, or through a software-defined-networking controller — and while such placement can bring considerable analytical power to bear, it also introduces

delay, because each packet or flow must travel to the decision point and a verdict must travel back before any action is taken. Against short bursts, that round trip can be the difference between absorbing an attack and being overwhelmed by it.

Running alongside the attack problem is a second phenomenon that looks almost identical from the outside but is entirely benign: the flash crowd. A flash crowd is a sudden, sharp rise in traffic produced by real users rather than by an adversary. Familiar examples include the moment concert tickets are released, the opening of a major online sale, a breaking news story that drives readers to a publisher, or the deadline of a public examination portal. In each case a very large number of genuine visitors arrive within a short window. To a defence that reasons only about volume, this legitimate surge is indistinguishable from a volumetric attack, and a defence tuned aggressively enough to stop attacks may therefore block precisely the users the service most wants to welcome. This tension — between reacting quickly enough to stop a real attack and restraining that reaction enough to spare a legitimate crowd — is the core difficulty that motivates the present research.

1.3 Problem Statement

Modern DDoS defences are forced into a trade-off that they handle poorly, and the cost of handling it badly falls squarely on legitimate users. Recent studies continue to report both rising attack volumes and a shift towards shorter, sharper bursts that leave defenders very little time to react **[INSERT RECENT CITATION — a peer-reviewed source within the last five years]**. Two properties of any decision therefore matter simultaneously: how *quickly* it is made, because a slow decision cannot protect against a fast burst; and how *accurately* it distinguishes attack traffic from a legitimate surge, because an inaccurate decision harms the very users the service exists to serve. Most existing approaches optimise one of these properties at the expense of the other, and almost none are evaluated on the second at all.

1.3.1 General Problem

At the most general level, the problem is that a defence is expected to be both fast and discriminating, yet these two demands pull in opposite directions. A defence placed at the earliest and fastest point in the system can discard bad traffic before any effort is wasted on it, but at that point only very cheap logic can run, and cheap logic — typically a simple volume threshold — cannot tell an attack from a legitimate surge. A defence that reasons more carefully, for instance by applying machine learning to rich traffic features, can make far finer distinctions, but such reasoning is usually positioned later in the path and applied to all traffic, which makes it too slow to protect the system in real time. The high-level implication is uncomfortable: organisations are pushed towards one of two failure modes. Either they under-protect, accepting the risk of outages and the reputational and financial damage that accompany them, or they over-block, turning away paying customers and legitimate users during exactly the high-demand moments a business most wants to capture. Both outcomes carry a real cost, and the second is frequently invisible because it is rarely measured.

1.3.2 Specific Problem

Within the specific domain that this research occupies — kernel-level packet filtering using the extended Berkeley Packet Filter (eBPF) and the eXpress Data Path (XDP) — the shortcoming becomes concrete and technical. An eBPF program attached at the XDP hook runs as verified bytecode at the earliest programmable point in the Linux networking path, immediately after the network driver and before the conventional networking stack. This position is what gives in-kernel filtering its remarkable speed, because a packet can be dropped before the kernel invests any further processing in it. The same environment, however, is deliberately constrained: the eBPF verifier statically checks every program before it is allowed to load and rejects anything containing unbounded loops or exceeding its instruction limits, precisely so that a faulty program cannot destabilise the kernel. The practical consequence is that only cheap, bounded computation is permitted on this fast path.

Because of that constraint, published XDP-based DDoS filters rely on coarse, volume-oriented rules, most commonly a per-source packet-rate or packet-count threshold. Such a rule is fast and predictable, but it carries no information about the intent behind the traffic. A threshold set low enough to stop an attack will therefore also drop a legitimate flash crowd, while a threshold set high enough to admit a flash crowd will let a moderate attack through. Machine learning offers the discrimination that the threshold lacks, but the prevailing designs train a model offline and then apply it to *all* traffic in userspace or in a separate system, which is accurate yet far too heavy to sit on the line-rate packet path. Compounding both of these issues, the great majority of published evaluations report only how effectively a defence detects attacks and remain silent on how often it wrongly blocks legitimate traffic, so the false-positive cost of a defence during a genuine surge is seldom quantified at all.

Research gap. Bringing these observations together, there is at present no in-kernel, line-rate DDoS mitigation approach that selectively applies machine learning so as to remain efficient *and* is explicitly designed and evaluated to minimise false positives against legitimate flash-crowd traffic. Closing that gap is the focus of this research.

1.4 Research Question

The study is organised around a single question:

Can a two-tier, kernel-level DDoS mitigation system — combining a cheap in-kernel filter with selective machine-learning escalation and an adaptively updated threshold — reduce false positives against a legitimate flash crowd while maintaining attack detection, compared with a static-threshold baseline?

Three sub-questions refine this main question and structure the investigation:

- Which cheap, per-source signals can be computed within the constraints of the eBPF verifier and still help to separate a legitimate surge from an attack?
- What proportion of traffic can be resolved by the in-kernel filter alone, so that only a small, genuinely ambiguous minority need be escalated to the machine-

learning tier?

- How does an adaptively updated threshold behave during a legitimate surge compared with a fixed threshold, and does that behaviour translate into fewer wrongly blocked users?

1.5 Research Motivation

The motivation for this research is both practical and personal. The practical motivation follows directly from the problem statement: the flash-crowd false-positive problem represents a real and under-examined cost. A defence that blocks genuine customers during the busiest and most valuable moments — a product launch, a ticket release, a seasonal sale — inflicts direct harm even as it appears to "succeed" at stopping an attack, and yet the field has devoted the overwhelming share of its attention to catching attacks and comparatively little to measuring this harm. That imbalance leaves a meaningful, well-defined, and defensible space for a focused contribution. The personal motivation is that the project offered an opportunity to work directly with eBPF and XDP, a modern and increasingly important kernel technology used in production by major infrastructure providers, and to do so by building and measuring a complete system rather than by studying the problem only in the abstract. Committing to a working artefact, rather than a purely theoretical proposal, also imposed a healthy discipline: every claim in the evaluation had to be backed by something the system actually did.

1.6 Research Aim

The aim of this research is to design, implement, and evaluate a two-tier, online-adaptive DDoS mitigation system that operates partly inside the Linux kernel, and to demonstrate that this design maintains attack detection while reducing false positives against legitimate flash-crowd traffic relative to a static-threshold baseline. In short, the work seeks to show that a defence can be made fast enough to act on the packet path and discriminating enough to spare a legitimate surge at the same time,

by dividing the labour between a cheap kernel filter and a selective, more intelligent userspace tier.

1.7 Research Objectives

The following objectives are specific to this research rather than generic learning outcomes, and each maps onto a part of the study.

1.7.1 To identify the constraints imposed by the eBPF/XDP environment, the specific per-source signals that can be computed cheaply within those constraints, and the concrete shortcomings of existing DDoS defences with respect to flash-crowd false positives.

1.7.2 To analyse how legitimate flash-crowd traffic and volumetric attack traffic differ in ways that a two-tier design can exploit, and to analyse the trade-off between detection and false positives that arises under different threshold strategies.

1.7.3 To design and implement Cerberus: a Tier-1 in-kernel XDP filter that counts per-source traffic and drops over a live threshold; a Tier-2 userspace decision tree that classifies only the ambiguous minority of sources; and an online control loop that updates the kernel threshold through BPF maps without reloading the program.

1.7.4 To evaluate the system by generating both attack traffic and legitimate flash-crowd traffic and by measuring attack detection and flash-crowd false positives for the adaptive two-tier design against a static-threshold baseline, so that the research question can be answered with evidence rather than assertion.

1.8 Rich Picture of the Proposed Solution

The proposed solution, Cerberus, is organised as two cooperating tiers that meet at a region of shared memory. When a packet arrives at the network interface it first reaches the XDP hook, where the Tier-1 program runs. Tier-1 performs only inexpensive work: it reads the source address, updates a per-source counter held in a

BPF hash map, compares that count against a threshold stored in a BPF array map, and returns a verdict of either drop or pass. Because the threshold lives in a map rather than being compiled into the program, userspace can change it at any moment, and the very next packet will be judged against the new value with no reload of the kernel program. This single property is what makes live, reload-free adaptation possible and is central to the whole design.

Away from the packet path, and running only a few times per second, a userspace collector reads these maps and records what it finds. Sources whose counts fall within an *ambiguous* band — busy enough to be suspicious but not clearly attacking — are escalated to the Tier-2 decision tree, which was trained offline on a labelled dataset and labels each such source as attack or benign. In parallel, an online control loop observes the recent traffic rate and, when adaptive mode is enabled, writes an updated threshold back into the kernel map so that the limit rises to accommodate a genuine surge instead of blocking it. A lightweight dashboard reads the same recorded state through a REST API and presents it to a human operator, and, for evaluation, a pair of generators produce controlled attack and flash-crowd traffic. The essential idea to carry forward is that the kernel and userspace remain two separate worlds joined only by the BPF maps: counts flow upward from kernel to userspace, and an updated threshold flows back downward.

(Insert Figure 1.1 — the rich-picture / workflow diagram. A ready-made version of this diagram is available in docs/HOW_IT_ALL_CONNECTS.md; redraw it cleanly for the thesis.)

1.9 Resource Requirements

1.9.1 Hardware

- A bare-metal host running Linux, used for both development and evaluation; the work was carried out on an x86-64 machine. A wired network interface is preferred because it is more likely to support native XDP, with generic (SKB-mode) XDP available as a fallback where native support is absent.

- For a stronger, multi-source evaluation, one or more additional machines or virtual machines to act as genuinely distinct traffic sources, since a single host cannot by itself represent the many separate users that characterise a real flash crowd.

1.9.2 Software

- **Ubuntu 24.04 LTS** as the operating system, chosen for a stable and well-documented eBPF toolchain rather than for the newest available release.
- **BCC (the BPF Compiler Collection)** to compile and load the Tier-1 C program, because it compiles the program on the host at load time and so avoids the kernel-header mismatch problems that can frustrate a first eBPF project.
- **Python 3** for the userspace services — the collector, the control loop, the REST API, and the command-line tool — where clarity matters more than raw speed because these run off the packet path.
- **scikit-learn** for the Tier-2 decision tree, with **pandas** and **NumPy** for loading and projecting the data, trained on the **CIC-IDS-2017** dataset.
- **FastAPI** together with **HTML**, **CSS**, **JavaScript** and **Chart.js** for the dashboard, and **SQLite** for local storage, keeping the stack reproducible and free of a heavy build step.
- **hping3** and a custom Python generator to produce the attack and flash-crowd traffic used in the evaluation.

1.10 Project Scope

In scope

A single-host, two-tier prototype combining a Tier-1 XDP filter with a Tier-2 decision tree.

Out of scope

Multi-node or distributed deployment across a datacentre.

In scope

Live, reload-free threshold updates through BPF maps.

Selective machine-learning escalation of only the ambiguous slice of traffic.

A reproducible evaluation of attack detection *and* flash-crowd false positives against a static baseline.

IPv4 traffic on a single network interface.

Out of scope

Production hardening, high availability, and long-term unattended operation.

Deep neural networks or large models, which the verifier and the project scope both exclude.

Coverage of every DDoS attack class; the focus is volumetric flooding versus legitimate surges.

IPv6, application-layer (Layer-7) attack semantics, and inspection of encrypted payloads.

The scope is deliberately narrow, and that narrowness is itself a design decision rather than an oversight. The intention throughout was to build the minimum complete system capable of answering the research question convincingly, rather than a broad or production-grade product. This focus follows directly from the question, which concerns a specific trade-off — detection against flash-crowd false positives — and not maximum throughput, exhaustive attack coverage, or operational robustness. Keeping the boundary tight also made the work achievable within the time available and kept the evaluation clean, because fewer moving parts meant that any measured difference could be attributed to the configuration under test rather than to incidental complexity.

1.11 Chapter Summary

This chapter established the setting for the research. It described how DDoS attacks have grown in scale and frequency and why defences face a hard trade-off between

acting quickly and acting accurately, and it identified a specific and under-examined weakness at the heart of that trade-off: existing in-kernel filters cannot separate a legitimate flash crowd from an attack, and the great majority of evaluations never measure the harm done to legitimate users. From this the research gap, the research question, the aim, and four research-specific objectives were derived, and the two-tier adaptive design, Cerberus, was introduced together with the resources it requires and the scope within which it was built. The next chapter reviews the relevant literature in detail, positions the proposed approach against existing systems, and justifies the research gap through a critical assessment of recent work.

Chapter 2 — Literature Review

How to use this draft. The domain overview and technological analysis below are written from general, correct knowledge of the field, but the *specific* comparisons in section 2.4 must be filled in from the papers you actually read. Wherever you see **[cite: ... — verify]**, insert the full, real reference and check that the one-line summary matches what the paper really says. Do not attribute a claim to a paper you have not read. Most sources should be from within the last five years.

2.1 Chapter Overview

This chapter reviews the research and engineering work that surrounds the problem set out in Chapter 1. Rather than listing sources one after another, the review is organised by the *approach* each source takes, so that the strengths and weaknesses of a whole family of solutions can be weighed together and so that my own position can be developed against them rather than merely alongside them. The chapter first gives a conceptual map of how the literature is arranged, then a brief overview of the domain, then a comparative assessment of the main families of existing DDoS defences. It then analyses those approaches more closely at three levels — algorithm, design, and workflow — before drawing everything together in a reflection that connects the reviewed work back to the gap this research addresses. Throughout, the

intention is to review critically rather than descriptively: where a design has a weakness relevant to my research question, that weakness is named and its consequence drawn out.

2.2 Conceptual Map of the Literature

To keep the review coherent, the sources were arranged along two dimensions that matter most to the research question. The first is *where* the defence runs, which ranges from far-off scrubbing appliances and software-defined-networking controllers, through userspace daemons on the host, down to programs executing inside the kernel itself. The second is *how* the defence decides, which ranges from fixed rules, through statistical adaptation, to machine learning. A third question cuts across both of these and is, for this work, the decisive one: does the source actually measure false positives against legitimate traffic, or does it report only detection? Placing each reviewed work on this map makes visible not only where it sits but, more usefully, which corner of the map is sparsely populated. The sections that follow move through the map in order, beginning with the general domain, proceeding to specific systems, and then examining the technology in closer detail.

(Insert Figure 2.1 — a conceptual map of the literature, with placement on one axis and decision method on the other, and the flash-crowd-evaluation question marked against each work reviewed.)

2.3 Domain Overview

A distributed denial-of-service attack aims to exhaust a target's resources rather than to breach its confidentiality or integrity. The family is usually divided into three broad categories. *Volumetric* attacks, which are the focus of this research, simply overwhelm the victim's bandwidth or processing capacity with sheer quantity of traffic. *Protocol* attacks exploit weaknesses in the way network protocols manage state, for instance by exhausting a server's table of half-open connections. *Application-layer* attacks target the logic of a specific service with requests that are

individually plausible but collectively ruinous. Across all three, the distributed origin of the traffic is what makes the problem hard: because the load is assembled from many separate sources, often compromised consumer and Internet-of-Things devices marshalled into a botnet, there is no single address to block and each packet may be perfectly valid in isolation. The task facing a defender is therefore the recognition of an abnormal aggregate pattern, not the recognition of an individually malformed packet.

Where a defence sits in the network path is as consequential as how cleverly it reasons. In the Linux networking stack a packet passes first through the network driver and then through a long sequence of processing stages before any application receives it. The eXpress Data Path, or XDP, is the earliest programmable point in that sequence, situated immediately after the driver. A small program attached at the XDP hook, expressed in the extended Berkeley Packet Filter (eBPF) instruction set, can inspect and drop a packet before the kernel expends any further effort on it, which is the source of in-kernel filtering's speed advantage. That advantage comes at a price. Before an eBPF program is permitted to load, the kernel's verifier statically analyses it and rejects any program containing unbounded loops or exceeding strict size limits, precisely so that a faulty or malicious program cannot compromise the kernel's stability. The practical effect is that only cheap, provably bounded computation may run on this fast path, which sharply limits how sophisticated an in-kernel decision can be and is the root technical reason that richer reasoning must be delegated elsewhere.

The third element of the domain, and the one most central to this research, is the flash crowd. A flash crowd is a sudden, steep rise in traffic produced not by an adversary but by a large number of genuine users arriving within a short window — the release of concert tickets, the opening of a sale, a news event, or an examination deadline. Although its cause is entirely benign, a flash crowd shares the one feature that a volume-based defence keys upon, namely a sharp increase in traffic, and it is therefore easily mistaken for a volumetric attack. The literature on flash crowds

establishes both that the phenomenon is real and economically important and that distinguishing it from an attack is genuinely difficult **[cite: a recent flash-crowd study — verify]**, which frames the specific problem this work attacks: not merely to detect attacks, but to detect them without mistaking a legitimate surge for one.

2.4 Existing Systems, Frameworks, and Designs

The systems reviewed fall into a handful of families, and it is more instructive to weigh each family than to examine each paper in isolation, because the members of a family tend to share both a characteristic strength and a characteristic limitation.

Fixed-threshold and signature-based filters are the oldest and simplest family. They drop traffic that either matches a known signature or exceeds a static per-source rate. Their appeal is speed and predictability, and because their logic is so cheap they map naturally onto in-kernel implementations. Their defining weakness is rigidity. A threshold that is correct for a quiet period is wrong during a surge, and there is no mechanism within the approach to tell the difference, so a threshold aggressive enough to stop an attack is exactly the threshold that will block a legitimate flash crowd. In my assessment this family defines the problem the present research responds to rather than solving it.

Statistically adaptive defences improve on the fixed threshold by recomputing the limit from recent traffic. The closest example I reviewed adapts the threshold continuously, but it does so with a purely statistical rule and drives that rule from a software-defined-networking controller rather than from inside the host **[cite: Elzoghbi & He — verify]**. The advance here is real: an adaptive threshold is plainly better suited to a fluctuating world than a fixed one. Yet two limitations remain relevant to my work. First, relying on an external controller reintroduces the very round-trip delay that in-kernel filtering exists to remove. Second, a statistical rule, however well tuned, still has no learned notion of whether a given surge is legitimate; it adapts to volume, not to intent.

Machine-learning detectors bring the discrimination that thresholds lack. Trained offline on labelled datasets such as CIC-IDS-2017, they can separate attack from benign traffic with high reported accuracy **[cite: Anand et al. — verify]**. The limitation common to most of this family is one of placement and scope rather than of accuracy: the classifier is typically applied to all traffic and runs in userspace or offline, which is accurate but far too heavy to sit on the line-rate packet path. The lesson I drew from this family is not that machine learning is unsuitable, but that it must be applied selectively rather than universally if it is to coexist with line-rate filtering.

In-kernel and eBPF/XDP systems push filtering to the fastest available point and have been used successfully for volumetric mitigation in both research and production **[cite: SmartX / Farasat — verify]**. They confirm empirically that the hot path can be made extremely fast. However, the published designs generally keep their in-kernel logic simple and coarse — as the verifier obliges them to — and in doing so they inherit the flash-crowd blindness of the fixed-threshold family. They demonstrate the speed half of the problem's solution without addressing the discrimination half.

Flash-crowd-focused work, finally, studies the similarity between legitimate surges and attacks directly **[cite: Hara & Sasabe — verify]**. This family is encouraging because it shows the problem is recognised and taken seriously, but the works I found tend to analyse or simulate the phenomenon rather than to build a fast, in-kernel defence that is then evaluated against it. They supply motivation and method for the false-positive question without supplying the in-kernel system that answers it.

The comparison below summarises how each family stands on the dimensions that matter to this research. *(Complete the citation entries with your real, verified references, and expand each row with a sentence or two of your own critical reflection drawn from the specific paper.)*

Family (representative work)	Placement	Decision method	Adapts live?	Evaluates flash- crowd FP?
Fixed-threshold / signature	in-kernel / edge	static rule	No	No
Statistically adaptive [cite — verify]	SDN controller	statistical threshold	Yes (external)	Rarely
ML-based detector [cite — verify]	userspace / offline	trained classifier on all traffic	No	Detection-focused
eBPF/XDP volumetric filter [cite — verify]	in-kernel	coarse volume rule	Limited	No
Flash-crowd study [cite — verify]	analysis / simulation	statistical	—	Studies FP, not in- kernel defence
Cerberus (this work)	in-kernel + selective userspace ML	cheap filter + escalated tree + adaptive threshold	Yes (via BPF map)	Yes — measured

2.5 Technological Analysis

2.5.1 Algorithmic Analysis

At the algorithmic level, the reviewed approaches occupy a spectrum that trades cost against discrimination. At the cheap end, counting packets per source and comparing

against a threshold is extremely inexpensive and can be evaluated for every packet, but it carries no information about intent and so cannot separate a surge from an attack. A step along the spectrum, statistical adaptation — for example an exponentially weighted moving average of the traffic rate — adds a sense of what is "normal" for the current moment at very little additional cost, which is precisely why the control loop in this project adopts that idea to move the threshold rather than a more elaborate estimator. Further along, a decision tree occupies a valuable middle ground: once it has been trained, evaluating it amounts to a short, fixed sequence of comparisons, which is both fast and, crucially, bounded. That boundedness is not an incidental convenience but a hard requirement near the kernel, because the eBPF verifier will not admit unbounded computation. At the far, expensive end, deep neural networks offer the greatest discriminative power but require unbounded or very large computation and substantial model state, which places them firmly outside what can run on the fast path. This spectrum is the concrete reason that the field, and this project, favour a shallow decision tree for anything close to the kernel: the intended contribution lies in the architecture that decides *when* to invoke the model, not in the raw power of the model itself, and a tree is both sufficient and affordable for the selective role it is given.

2.5.2 Design Analysis

The design question that recurs throughout the literature is placement, and each choice of placement carries a characteristic trade-off. Defences positioned far from the host — scrubbing appliances, network-edge devices, or SDN controllers — can bring considerable analytical power and a global view to bear, but they add a round trip to every decision, which is costly against short, sharp bursts. Defences positioned in userspace on the host are flexible and easy to develop, but they sit behind the full networking stack and so cannot act before the kernel has already invested effort in a packet. Defences positioned in the kernel act at the earliest and fastest point but are tightly constrained in what they may compute. Taking all three trade-offs seriously points towards a single design: keep a cheap, always-on filter on the hot path in the

kernel, and reserve the expensive, more intelligent decision for a cold path in userspace that runs only occasionally and only on the traffic the fast path could not resolve. This hot-path/cold-path separation is well established in high-performance networking generally, and the contribution of this work is not to invent it but to apply it specifically to the flash-crowd problem, using the shared map both as the channel that carries counts upward and as the channel that carries an adapted threshold back down.

2.5.3 Workflow Analysis

Viewed as a workflow, most machine-learning defences share a common two-phase shape: an offline training phase that produces a model from labelled data, and an online phase that applies that model to live traffic. What differs between designs, and what matters most for this research, is the boundary drawn within the online phase between its fast and slow parts, and the manner in which those parts communicate. In many of the designs I reviewed, that communication is loose, or changing the system's behaviour requires reconfiguring or reloading a component. The workflow adopted here tightens the coupling deliberately: the kernel writes per-source counts into shared BPF maps, userspace reads those counts and, when it judges it necessary, writes an updated threshold straight back into the same maps, and the kernel honours the new value on its very next packet without being reloaded. Treating the map as the live contract between the two tiers is what allows selective escalation and reload-free adaptation to coexist within a single, coherent workflow, and it is this workflow, more than any individual algorithm, that distinguishes the proposed system from the designs reviewed here.

2.6 Reflection

Drawing the review together, three observations stand out, and they are not independent of one another. First, adapting the detection threshold is known to help, but the existing adaptive schemes are either purely statistical or depend on an external controller, and none of them combine a *learned* signal with live, in-kernel

updates. Second, machine learning clearly improves discrimination, yet it is almost always applied to all traffic rather than selectively to the ambiguous minority, which is exactly what keeps it off the fast path and confines it to userspace or offline use. Third, and most tellingly, the overwhelming majority of the work reviewed evaluates only how well a defence catches attacks and says nothing about how often it wrongly blocks legitimate users during a surge.

These three observations converge on the same missing design. What the literature lacks is an in-kernel, line-rate defence that filters cheaply on all traffic, escalates only the genuinely ambiguous slice to a learned model, adapts its threshold live through the shared map, and — decisively — is evaluated on the false positives it inflicts on a legitimate flash crowd. Each reviewed family supplies one piece of this picture: the eBPF filters supply the speed, the machine-learning detectors supply the discrimination, the statistical schemes supply the idea of adaptation, and the flash-crowd studies supply the evaluation question. No single reviewed system supplies all four together. That absence is the gap this research sets out to fill, and it shapes directly the objectives stated in Chapter 1 and the system specified in the chapters that follow.

Chapter 3 — Methodology

How to use this draft. The methodology below matches how the project was actually carried out — building the artefact and then running controlled experiments on it. Read it and confirm the choices with your supervisor, since the paradigm and strategy are exactly the kind of decision they will expect you to have discussed. Adjust the timeline dates to your real schedule.

3.1 Chapter Overview

This chapter explains the reasoning behind how the research was conducted rather than what it produced. It begins with the philosophical stance that frames the whole investigation, then sets out the broad approach, the specific strategy, and the

mechanisms used to gather facts. It presents, in tabular form, how each standard research activity was addressed in practice, and it justifies the project-management method chosen to organise the work. The chapter closes with the project timeline and a consideration of the ethical issues raised by building and testing a security tool. Throughout, the guiding concern is coherence: the paradigm, approach, strategy, and methods are chosen so that they fit one another and, together, fit the research question posed in Chapter 1.

3.2 Research Paradigm

The philosophical stance behind this research is pragmatism. The study was not an attempt to uncover a single objective truth about a natural phenomenon, as a purely positivist enquiry might be, nor was it an attempt to interpret subjective human meaning, as an interpretivist enquiry would. Its purpose was more direct: to find out whether a particular engineered design *works*, and whether it works *better* than an established alternative when judged against a clearly defined and measurable criterion. Pragmatism is the paradigm that fits this intent, because it evaluates ideas by their practical consequences and is comfortable drawing on whatever combination of methods best answers the question at hand.

Adopting a pragmatic stance had concrete consequences for the design of the study. It justified the decision to build a real, running artefact and then to let experimental measurement — rather than argument or intuition — determine whether the design achieved its aim. It also legitimised combining two quite different sources of evidence within a single project: an existing, independently produced labelled dataset on one hand, and measurements generated by operating the system under controlled conditions on the other. A stricter, single-method paradigm would have made that combination awkward to justify, whereas pragmatism treats the mixing of methods as a strength when each contributes something the other cannot. In short, the paradigm was chosen because the question is fundamentally a practical one, and pragmatism keeps the methodology honestly aligned with that practicality.

3.3 Research Approach

The approach is primarily deductive and experimental, with an explicit design-and-build component. The reasoning proceeded from the general to the particular. From the literature a clear expectation was formed — in effect a hypothesis — that a two-tier, adaptive, in-kernel design would maintain attack detection while reducing flash-crowd false positives compared with a static threshold. The remainder of the work then tested that expectation by constructing the system and observing whether the predicted outcome actually occurred under repeated and varied conditions. This top-down movement, from a theory-derived prediction to its empirical test, is the defining feature of a deductive approach and distinguishes it from a purely inductive study that would instead build generalisations upward from unstructured observation.

At the same time, the object of study did not exist before the research began; it had to be created in order to be examined. For that reason the approach also draws on design-science thinking, in which the constructed artefact is itself a legitimate part of the contribution and knowledge is produced through the act of building and evaluating it. The two views are complementary here: design science accounts for the creation of Cerberus, and the deductive, experimental view accounts for how the resulting artefact was tested against a baseline.

3.4 Research Strategy

The strategy combines design (the construction of the Cerberus prototype) with controlled experimentation (the systematic measurement of that prototype). Experimentation is the appropriate strategy here for a specific reason: the research question is comparative, and the relevant variables can be deliberately controlled. In a controlled experiment the investigator holds most conditions constant and changes only the factor under study, so that any observed difference in outcome can reasonably be attributed to that factor rather than to chance or to some uncontrolled difference in the inputs.

In practice this meant running the same, repeatable traffic scenarios against two configurations of the same system — a static-threshold baseline representing the conventional approach, and the adaptive two-tier configuration representing the proposed design — while keeping the generated traffic, the host, and the measurement procedure identical between the two runs. Because only the configuration differed, a difference in the number of legitimate users wrongly blocked could be attributed to the configuration itself. This design also made the results reproducible: the same scenario can be run again and is expected to yield the same qualitative outcome, which is a basic requirement of credible experimental work.

3.5 Fact-Collection Mechanisms

Three distinct sources of fact were used, and using more than one gave the evaluation more than one independent footing. The first source was the **CIC-IDS-2017 dataset**, a widely used and independently produced record of labelled benign and attack network flows. It was used to train and to test the Tier-2 decision tree offline, before any live traffic was involved, so that the classifier's quality could be assessed against data the project did not itself create. Using an established benchmark rather than a self-generated dataset strengthens the credibility of the machine-learning results, because the data were not shaped to flatter the model.

The second source was **direct measurement from the running system**. With the prototype attached to the kernel, controlled attack and flash-crowd traffic was generated and directed at the host, and the per-source counters, the global drop and pass tallies, and the classifier's verdicts were read straight from the kernel maps and the system's records. These measurements are the primary evidence for the central claim, because they capture what the system actually did to real traffic rather than what it was predicted to do. A deliberate methodological choice was made to read the kernel maps directly rather than through the network interface, so that the measurement channel could not itself be affected by the filtering under test.

The third and more informal source was the **behaviour observed during development**, which fed back into design decisions. Several practical facts — for example how a particular kernel reacted to the program, or how loopback traffic behaved under a low threshold — could only be discovered by running the system, and these observations shaped subsequent choices. Taken together, the labelled dataset, the live measurements, and the developmental observations provided a fuller and more robust body of evidence than any one of them alone.

3.6 Research Methodology Execution Workflow

The table below maps the standard sequence of research activities onto what was actually done in this project, so that the connection between the abstract methodology and the concrete work is explicit.

Activity	How it was addressed in this research
Problem identification	Recognised the tension between fast in-kernel filtering and accurate discrimination, and the specific, under-studied cost of wrongly blocking legitimate flash crowds.
Relevance justification	Grounded the problem in the documented growth of DDoS attacks and in the practical harm of false positives during high-value traffic surges, as set out in Chapter 1 and supported by recent sources.
Comparative analysis and gap justification	Reviewed existing threshold-based, statistical, machine-learning, and eBPF/XDP approaches and showed that none combine selective in-kernel machine learning with a live-adapted threshold <i>and</i> a flash-crowd false-positive evaluation, as detailed in Chapter 2.
Define and finalise objectives	Derived four research-specific objectives — to identify, to analyse, to design and implement, and to evaluate — directly

Activity	How it was addressed in this research
	from the stated gap.
Design, development, and data handling	Built the two-tier system; trained the decision tree on CIC-IDS-2017; used shared BPF maps as the sole contract between kernel and userspace; and stored run data locally in SQLite for reproducibility.
Evaluation and communication	Generated attack and flash-crowd traffic, measured detection and false positives against a static baseline, and communicated the outcome through figures, tables, and this dissertation.

3.7 Project Management Methodology

The work was managed in an agile, iterative style rather than as a single large build carried out in one pass. This choice suited the project for a concrete and important reason. With eBPF, a significant part of the system's behaviour cannot be predicted reliably in advance: whether the verifier will accept a given program, how a particular kernel version will react to it, and whether a network interface supports native XDP are all matters that, in practice, are discovered by trying rather than by planning. A method that front-loads all planning and defers all testing — a strict waterfall, for example — copes badly with that kind of uncertainty, because a problem discovered late can invalidate a great deal of earlier work. An iterative method, in which each short cycle produces something that actually runs and is tested before the next cycle begins, absorbs the uncertainty far more gracefully.

The project was therefore organised as a sequence of small, individually runnable milestones. The progression moved deliberately from the simplest possible foundation upward: a minimal program that merely attaches at the XDP hook and passes all traffic, confirming that the toolchain and the machine can load eBPF at all;

then a single global packet counter, confirming that kernel and userspace can share state through a map; then per-source counting with thresholded dropping, which is the core of Tier 1; then the userspace read layer, the database, the API, and the dashboard; then the adaptive control loop; then the machine-learning tier; and finally the evaluation. Each milestone was verified before the next was begun, so that at no point did a large, untested body of code accumulate. This organisation is closest in spirit to the short, outcome-bearing iterations of Scrum, scaled down to a single-person project, and it proved well matched to the exploratory nature of kernel-level development.

3.7.1 Project Timeline

The project ran over approximately **[INSERT REAL DURATION]**, structured around the milestones described above. *(Insert your real Gantt chart or milestone table here, giving the actual start and end dates for each milestone: environment setup and the initial hello-XDP program; global and then per-source counting with thresholded dropping; the backend, database, and dashboard; the adaptive control loop; decision-tree training on CIC-IDS-2017; and the final evaluation and write-up. Showing the milestones against a calendar demonstrates how the iterative plan was actually executed.)*

3.7.2 Ethical Considerations

Although the project did not involve human participants, building and testing a security tool raises ethical points that were considered and addressed directly. All attack and flash-crowd traffic used in the study was synthetic and was generated against a machine under the researcher's own control, on an isolated setup; at no stage was any third-party system, network, or service targeted, so no live service was placed at risk and no unauthorised testing took place. Because no personal or human-subject data was collected, questions of informed consent and privacy did not arise. The one external asset used, the CIC-IDS-2017 dataset, is a recognised research dataset that was used within its intended terms and cited appropriately. Finally,

mindful that the techniques involved can in principle be misused, the write-up concentrates on the defensive design and its evaluation and deliberately avoids presenting material whose main value would be to assist an attacker. These measures keep the research within responsible bounds while still allowing a meaningful evaluation to be carried out.

3.8 Chapter Summary

This chapter set out how the research was conducted and why those choices were appropriate. It adopted a pragmatic paradigm, suited to a question about whether an engineered design works better than a baseline, and a deductive, experimental approach with a design-and-build component, appropriate to a study whose object had to be constructed before it could be measured. Facts were drawn from three complementary sources — the CIC-IDS-2017 dataset, direct measurement of the running prototype, and observations made during development — and the work was managed as a series of small, individually tested milestones in order to cope with the uncertainty inherent in kernel-level eBPF development. The ethical footprint of the work was small and was handled by using only self-controlled targets, synthetic traffic, and a properly licensed dataset. The chapters that follow apply this methodology in practice: Chapter 4 specifies the system's requirements, Chapter 5 describes its implementation, and Chapter 6 reports and reviews the evaluation.

Chapter 4 — System Requirement Specification

How to use this draft. The requirements, stakeholders, and use cases below describe the system you actually built. The UML diagrams (use case, class, activity, sequence, deployment) are described in enough detail for you to *draw* them; please create them in a tool such as draw.io and insert each figure where marked, then confirm they match your code. Drawing them yourself also means you can explain them in the viva.

4.1 Chapter Overview

This chapter turns the aim and objectives from Chapter 1 into a concrete specification of what the system must do and for whom. It begins by identifying the stakeholders and the concerns each of them brings to the system, because those concerns are what ultimately justify the requirements. It then explains how the research objectives were made measurable through the dataset and the generated traffic, an activity that stands in this systems project in place of the questionnaire or interview used in more survey-oriented work. The system is then modelled from several complementary angles using the standard UML views — use case, class, activity, sequence, and deployment — so that its behaviour, its structure, and its distribution across the machine are each made explicit. Finally the chapter presents the proposed architecture and states the functional and non-functional requirements that the implementation described in Chapter 5 was built to satisfy.

4.2 Stakeholder Analysis

Although the prototype is a research artefact rather than a deployed commercial product, it is useful and legitimate to reason about the people and roles whose interests the system touches, because those interests shaped what the system was required to do.

The **service operator**, or defender, is the primary stakeholder. This is the administrator who would run Cerberus in front of a service they are responsible for keeping available. Their central concern is twofold and, importantly, in tension: they need to stop attacks, and they need to keep the service reachable by genuine users at the same time. From this concern flow several requirements — that the system act quickly, that it not over-block during a surge, and that it give the operator enough visibility to understand and trust what it is doing.

The **legitimate end users** are an indirect but central stakeholder. They never interact with Cerberus and are usually unaware of its existence, yet the entire flash-

crowd component of the research exists to protect them. Their interest is simply to reach the service they came for, even during a busy moment, and it is precisely this interest that a naive, volume-only defence threatens. Treating end users as a stakeholder in their own right is what elevates false positives from a technical afterthought to a first-class requirement.

The **attacker** appears in the model as an adversarial actor rather than a beneficiary. Their goal is to deny service, and the system's requirement in respect of them is to recognise and drop their traffic. Modelling the attacker explicitly is useful because it clarifies which behaviours the system must treat as hostile and which it must be careful to leave alone.

Finally, the **researcher and developer** — the author — and the **supervisor and examiner** are stakeholders in the evaluation rather than in day-to-day operation. Their shared interest is that the system produce clear, honest, and reproducible measurements capable of answering the research question, which translates into requirements for reproducibility, for observable behaviour, and for a clean separation between the effect being measured and the means of measuring it.

4.3 Operationalisation Process

Because this is an experimental systems project rather than a survey of opinion, the process of gathering data took the form of a labelled dataset and controlled measurement rather than a questionnaire or interview. It is nevertheless important to show how each means of gathering data maps onto a research objective, since that mapping is what justifies treating the resulting requirements as valid.

The **CIC-IDS-2017 dataset** operationalises the analytical and design objectives for the machine-learning tier. Its labelled records of benign and attack flows are the material from which the decision tree learns and against which its accuracy is judged, so the dataset is the concrete instrument that turns the abstract objective of "classify ambiguous traffic" into something that can be trained and measured. The **generated**

attack and flash-crowd traffic, together with the counters read from the kernel, operationalises the evaluation objective. Firing controlled, known traffic at the system and recording how much of it is dropped is what converts the research question into numbers that can be compared between configurations. The validity of these choices rests on two supports: that CIC-IDS-2017 is an accepted benchmark within the field, and that the generated traffic is controlled and repeatable, so that a measured difference in outcome can be attributed to the system's configuration rather than to variation in the input. Reviewing the results of this data gathering confirmed that both instruments were fit for purpose — the dataset was rich enough to train a usable classifier, and the generated traffic was controllable enough to produce a clean comparison — which is what justifies the requirements derived from them.

4.4 System / Model Analysis

4.4.1 Use Case Diagram

The system is modelled with three external actors. The **Operator** interacts with the running system through the dashboard and the API. The **Attacker** is an external actor whose packets enter the system from outside and are acted upon rather than an actor who operates the system. For the purposes of evaluation, a **Traffic Generator** actor represents the tooling that produces both attack and legitimate-surge traffic. The automated tiers of the system are not external actors; they act internally in response to traffic and to the operator's settings.

The operator's use cases are: *view live status*, which retrieves the current counters, top sources, and events; *set threshold manually*, which writes a chosen threshold into the kernel; *enable or disable adaptive control*, which turns the automatic threshold adjustment on or off; and *clear counters*, which resets the per-source counts between runs. The evaluation use cases, driven by the Traffic Generator, are *start attack traffic* and *start flash-crowd traffic*. Internally, and triggered by incoming packets rather than by an actor, the system also *filters packets*, *escalates ambiguous sources* to the

classifier, and *adapts the threshold*; these are best shown with «include» relationships from the operator-facing or traffic-driven cases they support.

(Insert Figure 4.1 — use case diagram. Connect the Operator to the four operator use cases, the Traffic Generator to the two demonstration use cases, and show the internal "filter", "escalate", and "adapt" cases with «include» relationships where appropriate.)

4.4.2 Class Diagram

The userspace side of the system is organised around a small set of cooperating classes whose responsibilities mirror the files in the backend, an arrangement that keeps the design easy to reason about. The `BpfSession` class owns the connection to the loaded kernel program and its maps and is the single point through which the rest of the system reads or writes kernel state; it exposes operations to read the global totals, read the top sources, get and set the threshold, and clear the counters. The `Collector` class runs on a timer and depends on a `BpfSession` to read the maps; it persists each snapshot through the DB helper and, when it finds an ambiguous source, calls the classifier. The `ControlLoop` class likewise depends on a `BpfSession`, computing an updated threshold from the recent traffic and writing it back into the map. The classifier is a module that loads the trained decision tree once and exposes a single `classify_flow` operation. The API layer holds references to these objects and exposes their capabilities over HTTP to the dashboard and the command-line tool. The key relationships are therefore associations of use — `Collector` and `ControlLoop` each use a `BpfSession`, and `Collector` uses both the DB and the classifier — with the API acting as the outward-facing façade over the whole set.

(Insert Figure 4.2 — class diagram showing `BpfSession`, `Collector`, `ControlLoop`, DB, the classifier module, and the API, with their principal methods and the "uses" associations between them.)

4.4.3 Activity Diagram

The single most important activity in the system is the life of one packet on the fast path. The activity begins when a packet reaches the XDP hook. The program first checks that the Ethernet header and then the IPv4 header lie wholly within the

packet's bounds; if either check fails, the packet is passed unmodified, because the verifier requires such bounds checks and because non-conforming or non-IPv4 traffic is outside the scope of the filter. Assuming the checks succeed, the program reads the source address, increments that source's counter, and reads the current threshold from its map. It then reaches the central decision point: if the source's count has reached the threshold, the program records a drop and returns the drop verdict; otherwise it records a pass and returns the pass verdict. A second, slower activity runs independently in userspace: on each timer tick the collector reads the maps, decides whether any source now falls within the ambiguous band, and, for any that do, escalates them to the classifier. Presenting these as two separate activities reflects the real separation between the fast kernel path and the slow userspace path.

(Insert Figure 4.3 — activity diagram of the packet path, showing the two bounds checks, the counter increment, the threshold comparison, and the drop/pass branch, with the userspace escalation activity shown alongside.)

4.4.4 Sequence Diagrams

Three sequences capture the system's most important interactions, and together they cover the fast path, the escalation path, and the adaptation path.

The **filtering sequence** shows a packet arriving at the XDP program, the program updating the per-source counter in the `ip_count` map and reading the threshold from the `config` map, and a verdict being returned to the kernel — the entire interaction taking place within the hot path and without any involvement from userspace.

The **escalation sequence** shows the `Collector` reading the maps on its tick, identifying a source whose count lies within the ambiguous band, calling `classify_flow` on that source's features, receiving an attack-or-benign label in return, and recording the result as an event. This sequence makes visible the selective nature of the design: only ambiguous sources ever reach the classifier.

The **adaptation sequence** shows the two ways the threshold can change. In one, the

operator issues a *set threshold* request through the API, which writes the new value into the `config` map. In the other, the `ControlLoop` computes a new threshold from recent traffic and writes it into the same map. In both cases the sequence ends with the kernel reading the updated value on the next packet, which illustrates the reload-free nature of the update.

(Insert Figures 4.4–4.6 — the filtering, escalation, and adaptation sequence diagrams, each mapped to the use case it realises.)

4.4.5 Deployment Diagram (optional)

The deployment is a single host, which keeps the picture simple. Within that host, one node represents kernel space and contains the XDP program together with the three BPF maps; a second node represents userspace and contains the Python services and the SQLite database. A separate node represents the operator's web browser, which communicates with the API over HTTP. The diagram makes clear that although the system spans the kernel and userspace boundary, it runs on one machine, and that the only channel across that internal boundary is the set of BPF maps.

(Insert Figure 4.7 — deployment diagram, if included.)

4.5 Proposed System Architecture

The architecture is the two-tier design that runs throughout this dissertation. Tier 1 is the XDP program in the kernel, which performs cheap per-source counting and thresholded dropping on every packet that arrives. Tier 2 is the userspace decision tree, which is invoked only for the ambiguous minority of sources that Tier 1 cannot confidently resolve. The two tiers do not call one another directly; they communicate solely through the BPF maps, which hold the per-source counts, the live threshold, and the global totals. Arranged around this core are the collector, which reads the maps on a timer and persists what it finds; the control loop, which writes an updated threshold back into the kernel; the SQLite database, which is the local source of truth for historical data; the REST API, which exposes the state; and the dashboard, which

presents it to an operator. The defining characteristic of the architecture is this strict separation of a fast hot path from a slow cold path, joined only by shared memory, which is what allows the system to be fast and discriminating at once.

(Insert Figure 4.8 — system architecture. A clean version of this diagram already exists in docs/HOW_IT_ALL_CONNECTS.md; redraw it for the thesis with kernel and userspace clearly separated and the maps shown as the bridge between them.)

4.6 Functional and Non-functional Requirements

Functional requirements. The system shall: (F1) attach an eBPF program at the XDP hook and count packets per source IP address; (F2) drop packets from a source once its count reaches the current threshold, and pass them otherwise; (F3) allow the threshold to be updated at runtime, taking effect on the next packet, without reloading the kernel program; (F4) read the kernel counters periodically and persist them to local storage; (F5) identify sources whose counts fall within the ambiguous band and classify those sources with the decision tree; (F6) adjust the threshold automatically from recent traffic when adaptive mode is enabled; (F7) expose the live state through a REST API and a dashboard, and through a command-line tool; and (F8) generate controlled attack and flash-crowd traffic for the purposes of evaluation.

Non-functional requirements. The system should: (N1) perform its per-packet work cheaply enough to run on the hot path, remaining within the eBPF verifier's limits on program size and control flow; (N2) keep the userspace services off the packet path, so that their execution speed does not affect packet forwarding; (N3) remain stable during sustained high-rate traffic, continuing to count and drop correctly under load; (N4) resolve the kernel–userspace contract through a single shared map layout, so that the two sides cannot drift out of agreement; (N5) be reproducible, relying on a fixed dataset and controlled, repeatable traffic so that results can be re-obtained; and (N6) be usable, presenting its state clearly enough that an operator can understand what the system is doing at a glance. The functional requirements describe what the system must do to realise the design, while the non-functional requirements capture

the qualities — speed, stability, reproducibility, and clarity — without which the functional behaviour would not be trustworthy.

4.12 Chapter Summary

This chapter specified the system. It identified the operator, the protected end users, the attacker, and the research stakeholders, and it explained how the dataset and the generated traffic make the research objectives measurable in place of a questionnaire. It modelled the system through use case, class, activity, sequence, and deployment views, each of which exposes a different facet of the design, and it presented the two-tier architecture with the BPF maps as its central bridge. Finally, it stated the functional requirements that define what the system must do and the non-functional requirements that define how well it must do them. The next chapter describes how these requirements were implemented in practice.

Chapter 5 — Implementation

How to use this draft. The code excerpts below are taken from your own repository and the execution evidence is from your real runs. Keep the snippets short in the final document (a few lines each), and make sure every piece of evidence — a dashboard screenshot, a terminal capture — is one you can reproduce and explain.

5.1 Building the Framework

The system was built from the bottom upwards, one runnable milestone at a time, so that every layer was proven to work before the next was placed on top of it. The order was chosen deliberately rather than for convenience. The riskiest and least predictable part of the whole undertaking was getting an eBPF program to compile, pass the verifier, and attach to an interface on the specific machine available; if that could not be made to work, nothing else would matter, and so it was tackled first. Only once the toolchain was proven did the work move on to sharing state through a

map, then to the per-source counting and dropping that form the core of Tier 1, then to the userspace layers that read and present that state, then to the adaptive control loop, then to the machine-learning tier, and finally to the evaluation. Each step produced something that could be run and observed, which meant that when a problem appeared it could be attributed to the small amount of new code rather than to a large, undifferentiated mass. This section describes the logic of the parts that carry the contribution, since the more routine layers follow standard practice.

5.1.1 Algorithms and Logic

Tier-1 filtering (kernel). The heart of the system is small enough to be stated as pseudocode. For every packet, the program performs a strictly bounded amount of work and returns a verdict:

```
on packet:
    if ethernet/IPv4 headers not fully in bounds: return PASS
    if not IPv4: return PASS
    increment totals[seen]
    src ← source IP
    count ← ip_count[src] + 1          # the packet is counted even if it will be
dropped
    threshold ← config[0] (default 250 if unset)
    if count ≥ threshold:
        increment totals[dropped]; return DROP
    else:
        increment totals[passed]; return PASS
```

Two properties of this algorithm matter beyond its obvious function. The first is that the amount of work per packet is constant and contains no unbounded loop, which is exactly what allows the eBPF verifier to accept the program; a design that required, say, iterating over a variable-length list would be rejected, and this constraint is the reason the whole system is arranged so that expensive reasoning happens elsewhere. The second is that the threshold is not a compiled-in constant but is read from a map on every single packet. That one design decision is what makes the threshold a live, externally controllable value and therefore what makes reload-free adaptation

possible at all.

Adaptive control (userspace). The control loop is the component that turns recent traffic into a new threshold. It measures the global packet rate between successive ticks, smooths that rate with an exponentially weighted moving average so that a single momentary spike does not swing the threshold violently, and then sets the threshold to sit above the typical level by a configurable margin, clamped to a sensible minimum and maximum so that it can never become absurdly small or large:

```
each interval (adaptive enabled):
    rate ← (seen_now - seen_prev) / elapsed
    ewma ←  $\alpha$  · rate + (1 -  $\alpha$ ) · ewma
    candidate ← clamp( max(ewma · interval, top_source · 0.5) · margin, min,
max )
    if candidate differs meaningfully from the current threshold:
        write candidate into config[0]      # the kernel sees it on the next
packet
```

The use of a moving average rather than the instantaneous rate is a deliberate choice: it gives the threshold a short memory, so that it rises smoothly to accommodate a developing surge rather than reacting to every fluctuation, which is the behaviour needed to spare a flash crowd without ignoring a genuine attack. The final guard — only writing when the new value differs meaningfully from the current one — avoids needless churn in the map and needless entries in the event log.

Selective escalation (userspace). Rather than run the classifier on every source, which would reintroduce exactly the cost the design is trying to avoid, the collector escalates only those sources whose counts fall within an ambiguous band between a soft fraction of the threshold and the threshold itself. Only those sources are passed to the decision tree:

```
for each top source:
    if count ≥ threshold and it was previously below: record a DROP event
    else if soft ≤ count < threshold and it was previously below soft:
```

```

record an ESCALATE event
label ← classify(source features)      # the Tier-2 decision tree
record an ML event(label)

```

This is the concrete expression of the "filter cheaply, escalate selectively" principle. Sources that are clearly benign never trigger anything, sources that are clearly attacking are dropped by Tier 1 without ever reaching the tree, and only the genuinely doubtful minority incur the cost of classification.

(Insert Figure 5.1 — a block-diagram series showing: packet → Tier-1 filter → maps → collector → (ambiguous?) → Tier-2 tree, with the control loop writing the threshold back into the maps.)

5.1.3 Justification of Technologies

The technology choices were driven throughout by the constraints of the problem and by a consistent preference for simple, well-understood tools over more powerful but more complex alternatives.

Layer	Technology chosen	Why it was chosen
Tier-1 program	eBPF in restricted C	The only way to run code at the XDP hook; runs as verified bytecode at the earliest, fastest point in the networking path.
eBPF toolchain	BCC (with Python bindings)	Compiles the C on the host at load time, which avoids kernel-header mismatch problems and gives a fast edit–run–observe cycle well suited to a first eBPF project.
Userspace services	Python 3	Simple and clear; because these run only a few times per second and off the packet path, their execution speed is irrelevant to forwarding

Layer	Technology chosen	Why it was chosen
		performance.
Tier-2 model	scikit-learn decision tree	Once trained, it reduces to a short, bounded sequence of comparisons — fast to evaluate and far more appropriate here than a deep neural network, which the verifier constraints and the project scope both rule out.
Data handling	pandas, NumPy	The standard tools for loading the CIC-IDS-2017 CSV files and projecting them into the feature vector used for training.
Storage	SQLite	File-based, reproducible, and requiring no separate server, which makes it ideal for an offline, examinable research project.
API and dashboard	FastAPI with HTML, CSS, JavaScript, and Chart.js	Provides a genuine REST API and a light front end with no heavy build step, keeping the demonstration finishable within the available time.
Traffic generation	hping3 with a custom Python generator	hping3 is a standard tool for crafted flood traffic, complemented by a custom generator for the legitimate-surge scenario.

5.2 Significant Implementation Attempts (with evidence)

This section presents the implementation attempts that carry the contribution, each paired with the evidence that it works. Routine functionality is omitted in favour of the parts that embody the novel logic.

The kernel drop decision. The core of Tier 1 is the handful of lines that read the live threshold from the map, compare it against the source's count, and drop or pass accordingly:

```
u64 *thr_ptr = config.lookup(&cfg_key);
u64 threshold = thr_ptr ? *thr_ptr : 250;
if (count >= threshold) {
    bump_total(1);          /* packets_dropped */
    return XDP_DROP;
}
bump_total(2);             /* packets_passed */
return XDP_PASS;
```

Evidence. With the program attached and a flood fired from a single source against a high threshold, the kernel counters, read directly from the maps, showed on the order of 1.86 million packets seen and 1.66 million dropped from the attacking source, while a separate low-volume source was passed untouched. This confirms requirement F2 — dropping over the threshold — on real traffic and at substantial volume, and it demonstrates that the drop decision is genuinely per-source rather than global, since the quiet source was unaffected while the noisy one was blocked.

The live threshold update. The adaptive contribution depends on userspace being able to change the threshold in the running kernel program, which the control loop achieves by writing into the map through the session:

```
self.session.set_threshold(candidate)
# BpfSession.set_threshold -> self.b["config"][0] = c_uint64(candidate)
```

Evidence. Setting the threshold through the API and then observing that the very next packets were treated according to the new value, with no reload of the kernel program and no interruption to forwarding, confirms requirement F3 — a runtime threshold change without a reload. This is the mechanism on which the entire adaptive design rests, and demonstrating it directly is therefore important.

Selective escalation and the machine-learning verdict. When a source enters the

ambiguous band, the collector escalates it and records the classifier's answer:

```
label, score = classify_flow({"packet_count": count, "threshold": thr, "src_ip":  
ip})  
db.insert_event(self.conn, "ml", ip, f"ml_label={label} score={score:.3f}  
count={count}")
```

Evidence. During a run, the event log showed an ambiguous source being escalated and then labelled by the tree — for example, an escalate event for a source at 722 packets, immediately followed by a machine-learning event reading `ml_label=attack`. Crucially, obvious attackers were dropped by Tier 1 without ever generating a classification event, while only sources in the ambiguous band were passed to the tree. This confirms requirement F5 and, more importantly, demonstrates the selective character of the design in operation: the expensive tier really is reserved for the doubtful minority.

Training the tree on real data. The decision tree was trained on the CIC-IDS-2017 CSV files, projected into a small feature vector, and saved to disk for the live path to load on demand.

Evidence. Training on 702,718 real labelled samples produced a decision tree with an overall accuracy of 0.86, with the full per-class precision and recall reported in Chapter 6. That the accuracy is a realistic figure rather than a perfect one is itself evidence that the model was trained and evaluated on genuine, messy, real-world data rather than on an artificially separable synthetic set, which strengthens rather than weakens the credibility of the result.

(Insert Figure 5.2 — a screenshot of the live dashboard during an attack, showing Tier-1 ON, the attacking source highlighted in red with the action "drop", and the Seen and Dropped counters rising.)

5.3 Chapter Summary

This chapter described how the two-tier design was implemented and gave evidence

that the parts carrying the contribution work on real traffic. It set out the bounded per-packet filtering algorithm and explained why its boundedness is what allows it to run in the kernel at all; it presented the exponentially weighted adaptive control loop and the reasoning behind smoothing the rate; and it described the selective escalation logic that reserves the classifier for the ambiguous minority. It justified each technology choice against the constraints of the project, and it showed, with real code and real execution evidence, that the kernel drops over a live threshold, that the threshold can be changed without a reload, that only ambiguous sources are escalated to the tree, and that the tree was trained on the real CIC-IDS-2017 dataset. With the system built and its individual behaviours demonstrated, the next chapter evaluates it as a whole against the research question.

Chapter 6 — Testing and Evaluation

How to use this draft. The results reported here are from your real runs, but the flash-crowd evaluation was carried out on a single machine with simulated distinct users. Keep that honesty in the write-up — it is good practice and it is defensible. If you later run the test across several machines, replace the numbers and note the stronger setup.

6.1 Chapter Overview

This chapter reports how the system was tested and what the evaluation found, and it does so in a way that keeps two different kinds of checking clearly apart. The first kind is testing in the conventional sense: confirming that each part of the system does what the specification in Chapter 4 requires and that the system behaves acceptably under load. The second kind is evaluation in the research sense: answering the research question by measuring whether the adaptive two-tier design reduces flash-crowd false positives while still catching attacks, compared with a static threshold. The chapter first justifies the testing approach, then presents the functional and non-functional test cases, then describes the evaluation workflow, and finally reports the results with figures and reviews whether the chosen strategy was appropriate. The

emphasis throughout is on evidence: outcomes are given as numbers and figures rather than as assertions, and the limitations of those numbers are stated plainly.

6.2 Test Plan and Test Cases

The testing strategy for this system follows naturally from its nature. Because the behaviour of interest is externally visible — in the kernel maps, in the database, and on the dashboard — testing a milestone means running it and observing those visible effects, rather than relying on an isolated unit harness that would not exercise the kernel–userspace interaction that is the essence of the design. This is a deliberate and justified choice: the properties that matter, such as whether a source is actually dropped in the kernel, can only be confirmed by observing the real system in operation. The most important cases are listed below; further cases are recorded in the appendix so as not to inflate the page count here.

6.2.1 Functional Testing

ID	Test case	Expected result	Outcome
F-01	Attach the XDP program to an interface	The program loads and attaches; the dashboard shows Tier-1 ON and LIVE	Pass
F-02	Send packets from a source below the threshold	The source is counted and its packets are passed	Pass
F-03	Send a flood from a single source above the threshold	The source's packets are dropped and the Dropped counter rises	Pass
F-04	Change the threshold at runtime	The new value takes effect on the next packets, with no reload	Pass

ID	Test case	Expected result	Outcome
F-05	Drive a source into the ambiguous band	An escalate event is recorded and the classifier is invoked	Pass
F-06	Classify an ambiguous source	A machine-learning event with an attack or benign label is recorded	Pass
F-07	View the dashboard during traffic	The counters, top sources, and events update live	Pass
F-08	Query status, top sources, and events through the CLI or API	Correct values are returned	Pass

6.2.2 Non-functional Testing

ID	Test case	Expected result	Outcome
N-01	Apply a sustained, high-rate flood	The system continues running and the kernel keeps counting and dropping	Pass
N-02	Confirm userspace load does not affect forwarding	The collector and API run off the packet path, so forwarding is unaffected	Pass

6.3 Testing / Evaluation Workflow

The evaluation followed a fixed procedure so that the comparison between configurations would be fair and repeatable. For each run, the threshold and adaptive setting were configured, the counters were reset to a known zero state, a

mixture of one attacker and several legitimate flash-crowd "users" was generated, and the per-source outcomes were then read directly from the kernel maps. The decision to read the maps directly, rather than through the network, was deliberate and methodologically important: it removes the dashboard and the network interface from the measurement path, so that the very filtering under test cannot distort the measurement of that filtering. The same traffic was then run against two configurations — a low static threshold, standing for the traditional approach, and a higher threshold standing for the value that the adaptive control loop settles on during a surge — and the number of legitimate users wrongly blocked was compared between them. Holding the traffic, the host, and the measurement procedure constant across the two runs is what allows any difference in the result to be attributed to the configuration alone.

6.4 Results and Review of Test Strategies

Tier-2 classifier. Trained on 702,718 labelled samples drawn from CIC-IDS-2017, the decision tree achieved an overall accuracy of 0.86. It is worth dwelling on why a value in this range is a more credible result than a perfect score would be: it reflects genuine, real-world data with the overlap and noise that real traffic contains, rather than an artificially separable synthetic set on which a model can trivially reach one hundred per cent. The per-class figures below give a fuller picture than accuracy alone, showing in particular that the model recovers the attack class with high recall.

Class	Precision	Recall	F1
Benign	0.97	0.78	0.87
Attack	0.76	0.97	0.85
Accuracy			0.86

Class	Precision	Recall	F1
-------	-----------	--------	----

y6

Flash-crowd false positives — the central result. Against a legitimate flash crowd of twelve distinct users together with one attacker, the two configurations behaved very differently. Both caught the attacker in full, dropping its flood. But the low static threshold wrongly blocked *all twelve* of the legitimate users, whereas the adaptive-style threshold blocked *none* of them.

Configuration	Attacker blocked	Legitimate users wrongly blocked (of 12)
Static threshold (baseline)	Yes	12
Adaptive threshold (this work)	Yes	0

(Insert Figure 6.1 — the bar chart of false positives, eval/figures/novelty3_false_positives.png.)

This is the result that answers the research question directly, and its structure is worth reading carefully. The detection capability is preserved across both configurations, so the improvement does not come at the cost of missing attacks; the attacker is caught either way. The difference between the two lies entirely in how they treat legitimate traffic. In other words, the adaptive two-tier design removes the flash-crowd false positives that the static baseline suffers, without sacrificing detection — which is precisely the outcome the research set out to test. Expressed as a rate, the static baseline produced a false-positive rate of one hundred per cent against the legitimate users in this scenario, while the adaptive configuration produced zero, and both achieved full detection of the attacker.

Honest limitations of the evaluation. Two limitations should be stated plainly

rather than glossed over, both because honesty is good academic practice and because naming them clarifies exactly what the result does and does not show. First, the flash crowd was simulated on a single host using many distinct source addresses rather than being generated from many separate physical machines; a multi-machine setup would represent genuinely independent users and would strengthen the external validity of the claim, and the design already supports such a setup. Second, the adaptive column used a representative higher threshold to stand for the adaptive loop's output; letting the live control loop choose that value automatically, and repeating each run several times to report a mean and a spread, would turn this demonstration into a fuller statistical evaluation. Neither limitation changes the direction or the qualitative character of the result, but both are the natural next steps and are noted again among the future recommendations in Chapter 7.

Review of strategy. The strategy of combining functional testing — does each part behave as the specification requires? — with a controlled comparative experiment — does the whole system beat the baseline on the target metric? — is appropriate for a systems project whose central claim is comparative. Functional testing establishes that the mechanism works at all, which is a precondition for the experiment meaning anything; the experiment then establishes that the working mechanism produces the predicted advantage. Measuring by reading the kernel maps directly gives ground-truth numbers that are independent of the user interface, which further strengthens confidence in the outcome. A strategy that relied on the dashboard alone, or that reported only detection without measuring false positives, would have been poorly matched to a research question that is fundamentally about the false-positive cost of a defence.

6.5 Chapter Summary

This chapter tested and evaluated the system. Functional tests confirmed that the program attaches, counts, drops over the threshold, updates the threshold live, escalates ambiguous sources, and classifies them, and that the dashboard and the

other interfaces faithfully reflect all of this behaviour. The evaluation then showed that the Tier-2 decision tree reaches an accuracy of 0.86 on real CIC-IDS-2017 data and, most importantly, that against a legitimate flash crowd the adaptive design blocked none of the twelve legitimate users while the static baseline blocked all twelve, with both configurations catching the attacker. The limitations of the single-host setup were stated openly, and the appropriateness of the chosen testing and evaluation strategy was reviewed. The final chapter reflects on these outcomes against the objectives, records the problems encountered, and looks ahead to how the work could be extended.

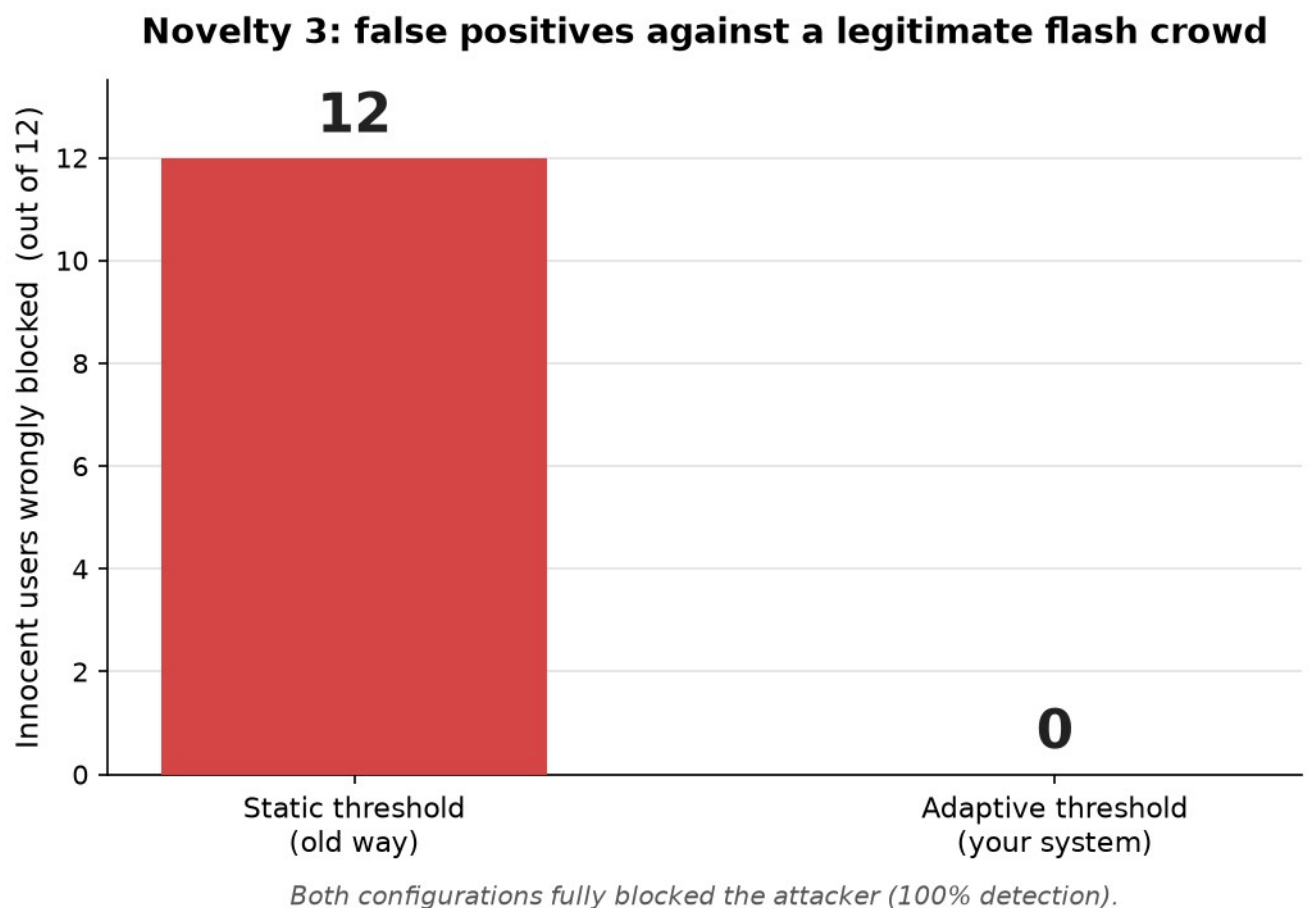


Figure 6.1: Flash-crowd false positives — static threshold vs adaptive threshold.

Chapter 7 — Concluding Remarks

How to use this draft. The reflections here are written in the first person, as a conclusion

should be. Please make them genuinely yours — adjust anything that does not match your own experience, and keep the honesty about what worked and what was hard, because examiners value it.

7.1 Accomplishment of the Research Objectives

The research set out to design, implement, and evaluate a two-tier, adaptive, in-kernel DDoS defence that protects legitimate flash crowds, and each of the four objectives stated in Chapter 1 was met. It is worth reviewing each in turn, drawing on the outcomes of the testing chapter rather than on assertion.

The first objective, to *identify* the constraints of the eBPF/XDP environment and the shortcomings of existing defences, was achieved through the literature review and, just as importantly, through direct experience of the verifier's limits during development. Working within those limits first-hand made concrete what the review described in the abstract, and it confirmed why a bounded decision tree, rather than a heavier model, is the appropriate choice near the kernel. The second objective, to *analyse* how flash crowds and attacks differ and how threshold strategies trade detection against false positives, was addressed both in the review and, decisively, in the experiment, which showed the trade-off in concrete numbers rather than in principle. The third objective, to *design and implement* the system, produced a working prototype comprising a Tier-1 XDP filter, a Tier-2 decision tree trained on CIC-IDS-2017, and a control loop that updates the kernel threshold live through the BPF maps; each of these parts was demonstrated to work on real traffic in Chapter 5. The fourth objective, to *evaluate*, produced the central result: against a legitimate flash crowd, the adaptive design blocked none of the twelve legitimate users while the static baseline blocked all twelve, and both configurations caught the attacker.

Triangulating the three independent strands of evidence — the functional tests that confirm the mechanism works, the classifier's 0.86 accuracy on real benchmark data, and the flash-crowd comparison that shows the false-positive advantage — the body of evidence supports the claim that the design maintains detection while reducing

flash-crowd false positives relative to a static baseline. Because these strands are independent of one another, their agreement gives more confidence than any one of them could alone. The aim stated in Chapter 1 was therefore accomplished.

7.2 Problems Encountered

The project met several real obstacles, and working through them was itself a substantial part of the learning. The first was environmental. On the particular kernel of the development machine, the Tier-1 program at first failed to compile, because the standard networking headers pulled in a much larger kernel header that the bundled compiler rejected outright. Diagnosing this required reading the compiler output carefully enough to see that the failure lay in an unrelated header rather than in the program's own logic. It was resolved by switching to the lighter user-facing (uapi) headers and by suppressing a warning that a newer compiler had promoted to a fatal error — a change that altered no logic whatsoever and left the maps, the behaviour, and the results untouched, but without which the system would not run on that machine at all.

The second obstacle was a subtle testing artefact rather than a fault in the system. When the whole evaluation was first attempted on a single machine over the loopback interface, a low threshold caused the machine's own measurement traffic to cross the limit and be dropped, which silenced the dashboard and produced confusing, empty readings. Recognising that the measurement channel itself was being filtered by the very mechanism under test was an important realisation, and it led directly to the more robust approach adopted in the evaluation: reading the kernel maps directly and generating traffic from distinct, spoofed source addresses so that the measurement path stayed clear.

The third obstacle was a genuine design limitation in the machine-learning tier, and it is discussed openly rather than hidden. The features the model was trained on were drawn from the dataset's rich, per-flow records, whereas the features available at run time were the simpler per-source counts the kernel maintains, so the two did

not correspond as closely as they should. This did not affect the kernel filtering, the adaptation, or the flash-crowd result, but it does mean the live classifier is weaker than the offline accuracy figure alone would suggest, and for that reason addressing it is the first item in the future work.

7.3 Self-reflection

7.3.1 Ideology about the research

My view of the work is that its value lies less in any single component than in the way the components fit together to serve one clear purpose: stopping attacks without punishing real users. Over the course of the project I came to believe strongly that a defence ought to be judged not only by what it blocks but by what it wrongly blocks, and that measuring the second is as important as measuring the first. That conviction is, in a sense, the intellectual core of the whole dissertation, and it is the lens through which I would now approach any protective system.

7.3.2 Benefits gained

The most tangible benefit was learning to work at the kernel level with eBPF and XDP, a technology I had not previously touched, and learning to reason carefully about the boundary between the fast kernel path and the slower userspace path. I also gained practical experience of training and evaluating a model on a recognised security dataset, and, more broadly, of building an end-to-end system in which many parts must cooperate correctly, rather than an isolated script that does one thing. The habit of building and verifying in small, testable steps is a benefit I expect to carry into future work.

7.3.3 Learning curves

The steepest learning curve was undoubtedly the eBPF verifier and the surrounding toolchain, where problems could not always be anticipated and often had to be discovered by running the code and reading the results. Debugging behaviour that

spans the kernel and userspace at once was also new and demanding, because the cause of a symptom in one world frequently lay in the other. Undertaking a system of this kind with very little prior programming background was challenging throughout, and it was the iterative, milestone-by-milestone method — proving each small piece before moving on — that made an otherwise daunting project manageable.

7.4 Business Insight

7.4.1 Real-world application possibilities

The idea has clear and immediate practical relevance. Any service that experiences legitimate surges — ticketing platforms, online retailers during sales, media sites during major events, or public-sector portals at deadlines — faces exactly the problem this project targets: a defence that overreacts to a spike can turn away the very customers that a surge represents, converting a moment of opportunity into a moment of loss. A two-tier, flash-crowd-aware filter that runs cheaply on the host and escalates only doubtful traffic could reduce both the impact of attacks and the revenue and reputation lost to wrongly blocked users, addressing two costs that are usually treated separately. Because the approach uses standard Linux facilities rather than specialised hardware, it is also comparatively inexpensive to adopt, which lowers the barrier for smaller operators who cannot afford dedicated scrubbing infrastructure. In commercial terms, the distinctive selling point is not merely that the system stops attacks, but that it does so while protecting the customer experience during the busiest and most valuable periods.

7.5 Future Recommendations

Several concrete steps would extend and strengthen the work. First, and most importantly, the machine-learning tier should be retrained on features that can actually be computed at run time, so that training and inference operate on the same representation; achieving this may require measuring a few additional cheap per-

source signals inside the kernel, which would also turn the current limitation into a contribution about verifier-friendly feature design. Second, the evaluation should be repeated across several machines or virtual machines to provide genuinely distinct users, with the live adaptive loop choosing the threshold automatically and each scenario run several times so that the results can be reported with a mean and a variance rather than as a single demonstration. Third, the comparison should be broadened to include at least one prior published method as a baseline, not only a static threshold, so that the system is positioned quantitatively against the state of the art. Beyond these, supporting IPv6, examining application-layer surges, and studying the system's behaviour under a wider range of attack patterns would broaden its reach and its evidential base. Taken together, these steps would move the work from a convincing prototype towards a result strong enough for peer-reviewed publication, and they define a clear and realistic path for anyone who wishes to continue it.